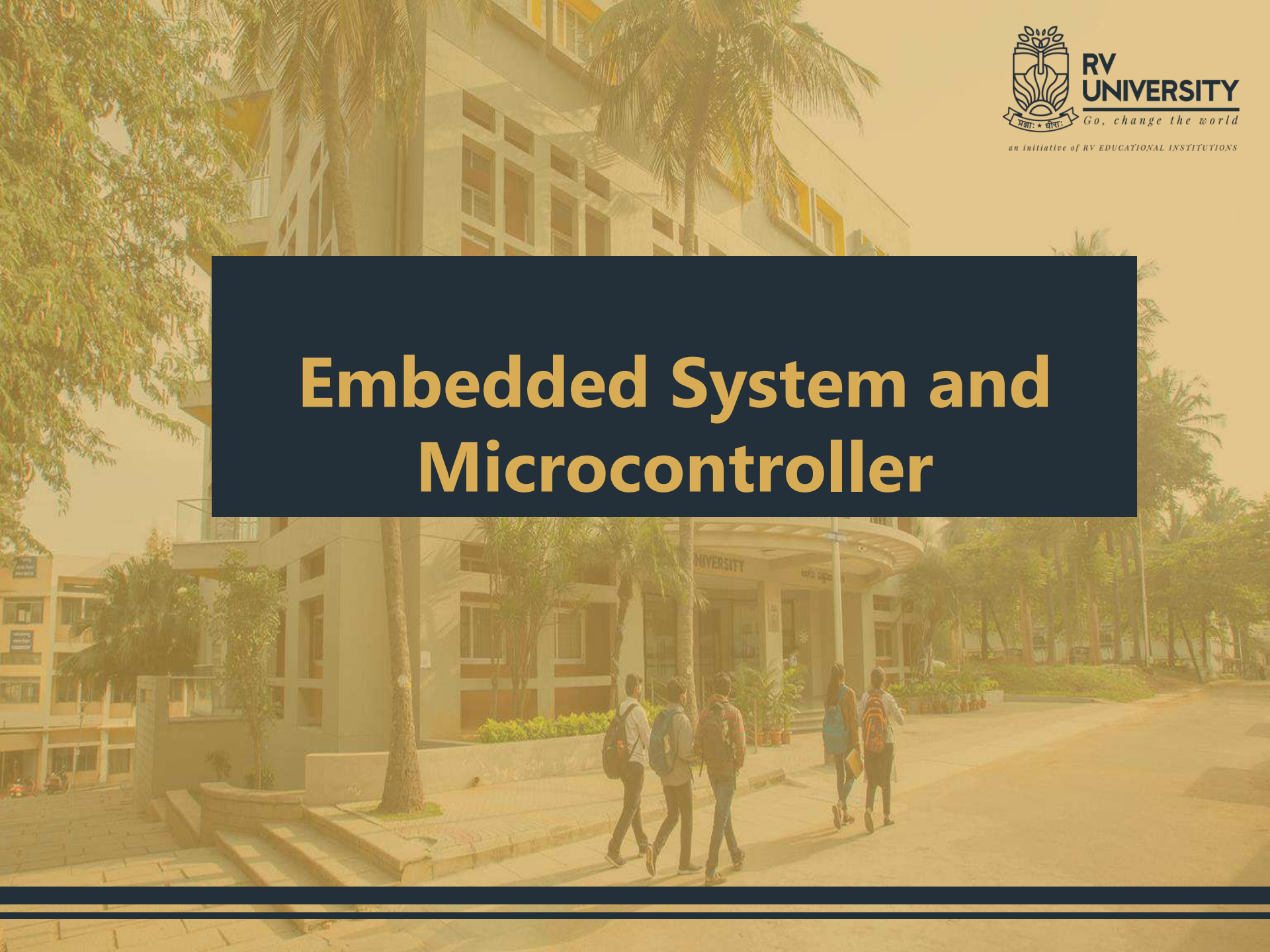


Embedded System and Microcontroller

Unit1

Introduction to Embedded Systems and Memory

Mouli Sankaran

Embedded Systems Overview

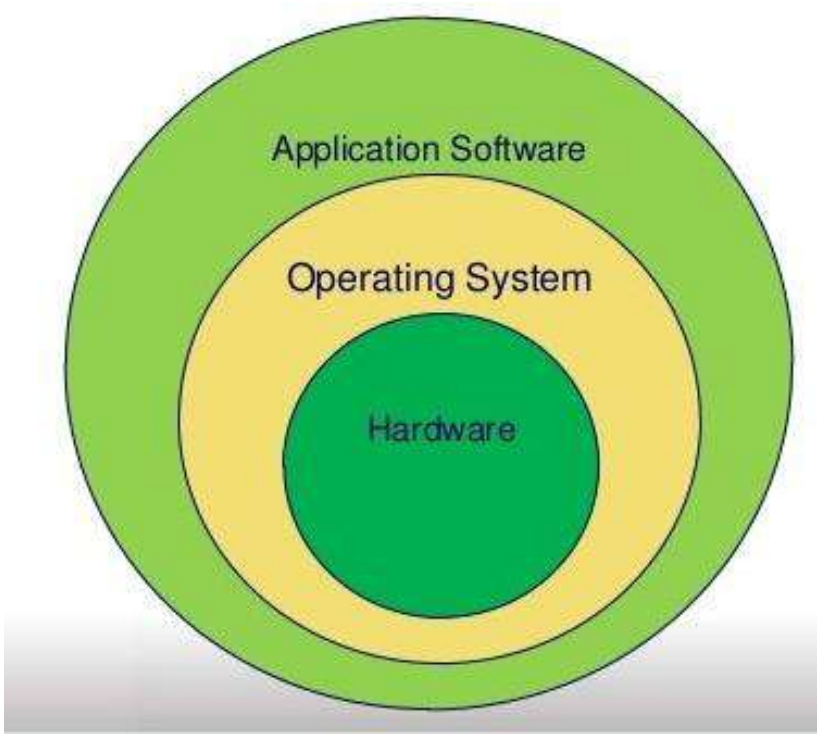
System:

- A system is an arrangement in which all its units are assembled to work together according to a set of rules.
- It can also be defined as a way of working, organizing or doing one or more tasks according to a fixed plan.
- **Example**: A digital watch is a system that displays the current time

Embedded System:

- As the name suggests, embedded means something that is attached to another thing.
- An embedded system can be thought of as a computer system having both hardware and software embedded in it.
  - An embedded system can be an independent system or it can be a part of a larger system.
- An embedded system is a microcontroller or microprocessor based system which is normally battery operated and designed to perform a specific task.
  - **Examples**: Fire alarms, coffee dispenser, smart door lock, flight controller, etc.

Embedded Software Architecture



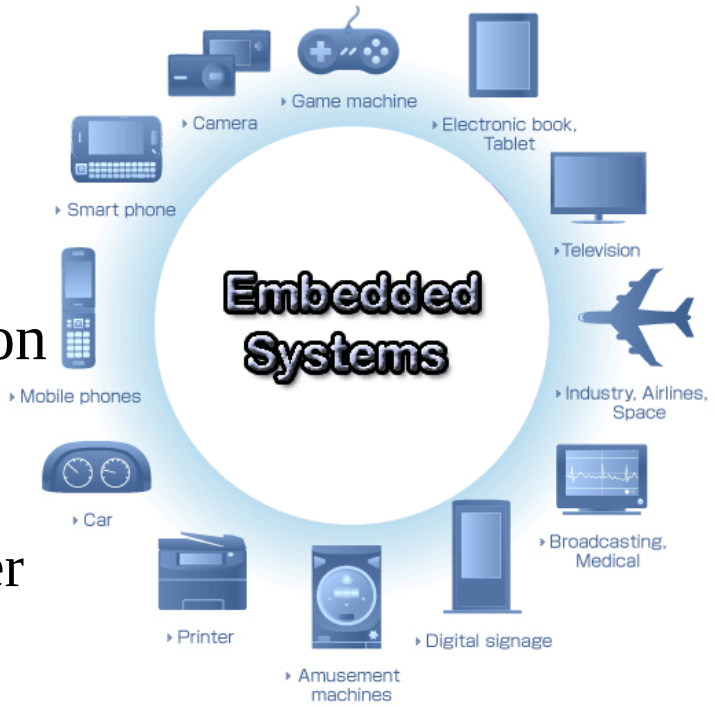
- OSs running on embedded systems are normally called RTOS, having stringent constraints on execution time, memory, latency, performance, etc.

RTOS: Real Time Operating System

Embedded Systems: Explained

An **embedded system** normally has **three components**:

- It has **hardware**.
- It has **embedded application software**.
- It has **RTOS** that supervises the application software and provides necessary support.
  - RTOS provides mechanisms to let the processor run processes in the system as per some scheduling algorithm to achieve the **latency requirements.**
  - RTOS defines the way the system works.
    - It sets the rules during the execution of application program.
  - A small scale embedded systems may not have an RTOS running in the system too



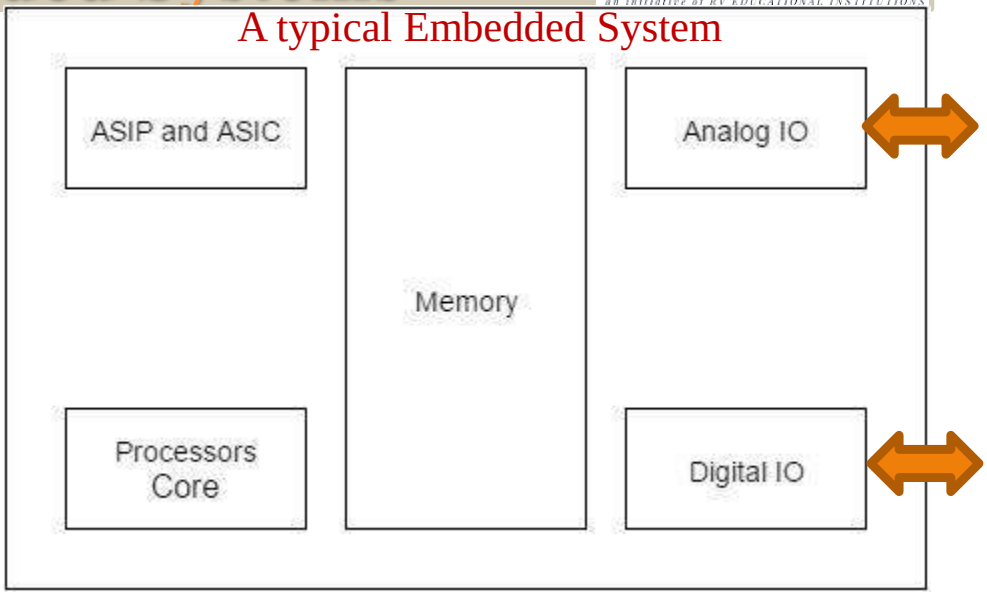
Bare metal is a computer system without a base operating system (OS) or installed applications.

Bare metal means a system where the **software** or **firmware** is running directly on the HW (no OS)

Characteristics of Embedded Systems

1. Characteristics of Embedded Systems

- **Single-functioned** – An embedded system usually performs a specialized operation and does the same repeatedly.
- **For example**: A pager always functions as a pager.



- **Tightly constrained** – All computing systems have constraints on design metrics, but those on an embedded system can be especially very tight, means missing the deadline would be disastrous.
- **Design metrics** is a measure of an implementation's features such as its cost, size, power, latencies and performance.
  - It must be of a size to fit on a single chip/board, must perform fast enough to process data in real time and consume minimum power to extend battery life.

ASIC: Application Specific Integrated Circuit

ASIP: Application Specific Instruction Processor

ASSP: Application Specific Standard Processor

Ref: ASIC Vs ASSP

2. Characteristics of Embedded Systems

- **Reactive and Real time** − Many embedded systems must continually react to changes in the system's environment and must compute certain results in real time without any delay.
  - Consider an example of a car cruise controller; it continually monitors and reacts to speed and brake sensors.
  - It must compute acceleration or de-accelerations repeatedly within a limited time; a delayed computation can result in failure to the control of the car.
- **Microprocessors based** − It must be microprocessor or microcontroller based on which the application or control software runs on top of an RTOS.

3. Characteristics of Embedded Systems

- **Memory** – It must have a memory, as its software usually embeds in Flash/ROM.
  - It uses DRAM or SRAM as Main Memory, normally no memory management done
  - It lacks full-fledged virtual memory support due to timing constraints similar to general purpose CPUs on Computers
  - It does not use any secondary memories (HDD) in the system, due to reliability issues and higher power consumption.
  - Flash memory is used instead of HDD for secondary storage.
- **Connected** – It must have to be interfaced to different peripherals, for connectivity or to process input and output.
  - For example, IoT devices should be connected to network.
- **HW-SW systems** – Software is used for high-end features, flexibility and configurability.
  - Hardware is used for higher performance and security as well.

Embedded Systems: Features and Challenges

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Features:

- Easily customizable (adaptable to environment)
- Low power consumption (for a longer battery life)
- Low cost (selling price needs to be cheaper)
- Enhanced performance (need to meet real-time constraints)
- Highly reliable in operation (safer operation)

Challenges:

- High development effort (effort involved in making it)
  - Because they need to be highly reliable
- Larger time to market (time taken to design it)
- Higher volume requirements (no. of units)
- Low power operations (for longer battery life)

Computer Systems

Hierarchical Levels of Computer Systems

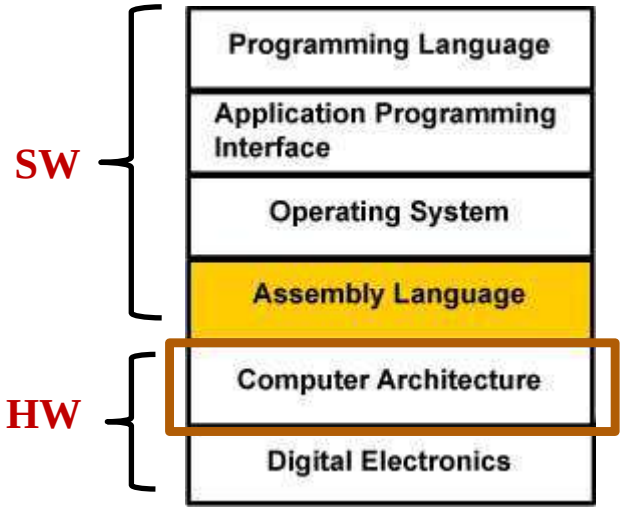


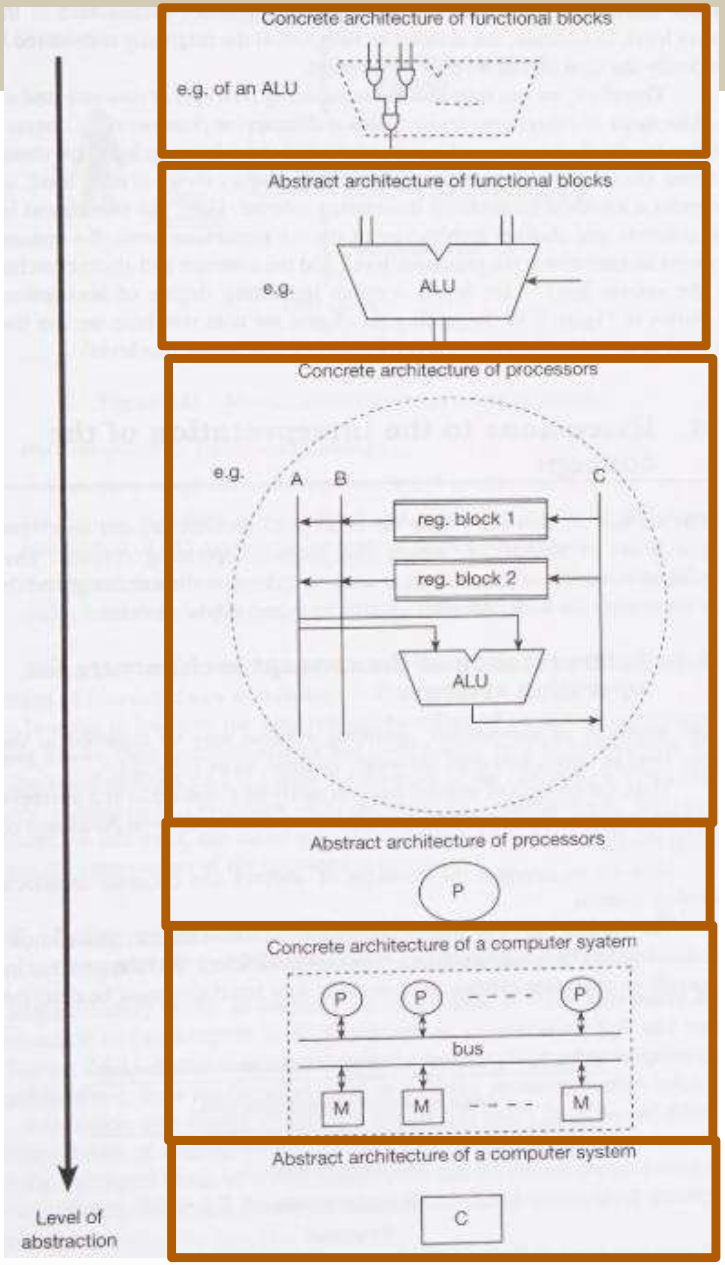
Table 2.2 A possible hierarchical six-level description of digital systems.

	Level of abstraction	*Basic components to be considered*	*Behavioural description by*
Computer Architecture	OS	OS	OS commands
	Computer system	Computer system	Instructions
	Processor	Processor	Basic instructions
	Functional block	Registers/MUXs, ALUs, micro-sequencer	Operations (register transfers, state sequencing)
	Circuit	Gates, FFs	Boolean equations
	Circuit element	Transistors etc.	Differential equations

Let us see different levels of abstractions of computer architecture next …

Ref: Levels of computer Systems

Architecture Levels: Abstraction



Logic Level

Functional Block Level

Architecture Level

System Level

Increasing levels of abstraction

Our focus in this course is here …

Computer/Processor Architecture

(Definitions)

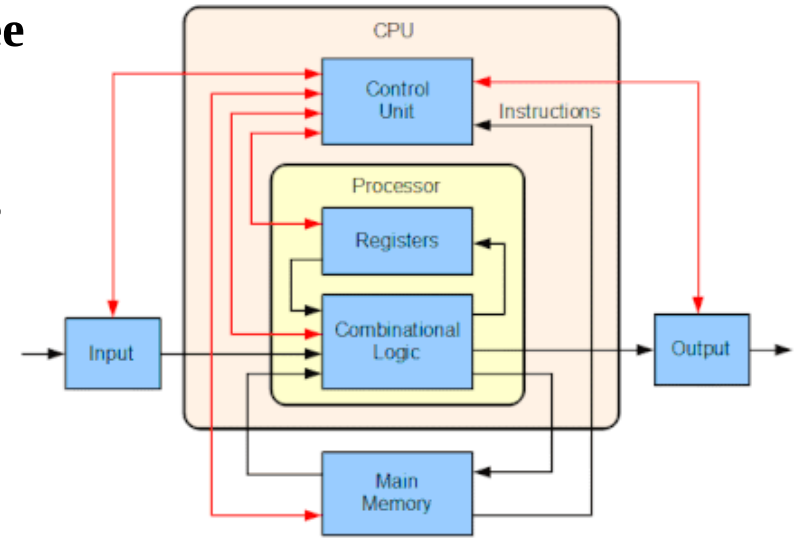
Computer Architecture: Definition

- **Computer architecture** is a set of rules and methods that describe the functionality, organization, and implementation of computer systems.

The **discipline** of **computer architecture** has **three main subcategories**

1. **Instruction set architecture** (**ISA**): It **defines** the machine code that a processor reads and acts upon.
   - As well as the word size, endianness, memory address modes, processor registers, data type, etc.
   - **Example**: x86, ARM, MIPS, etc.
2. **Microarchitecture**: It is also known as "computer organization".
   - This describes how a particular processor will **implement** the **ISA**.
   - **Note**: x86, converts internally all CISC type x86 instructions into RISC type before executing them.

- **Systems design**: Includes all of the other hardware components within a computing system, other than the CPU.
  - Cache, memory, internal bus, peripherals, etc.



- Block diagram of a basic computer with a single CPU.
- The **black lines** indicate the **data flows**, whereas the **red lines** indicate the **control flows**.
- Arrows indicate the direction of flows.

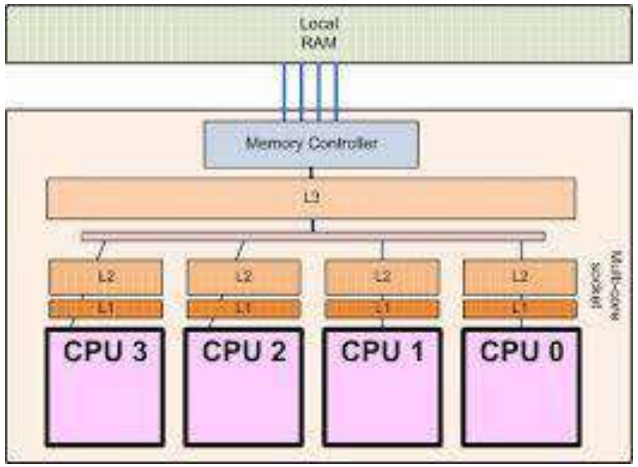
MIPS: Microprocessor without Interlocked Pipelined Stages

Processor Architecture: Definition

- **Processor Architecture:**
  - It details the design of processors (multi-cores as well), its internal organization, its interface with memory and peripherals.

Processor architecture elaborates the following:

1. **Instruction set architecture (ISA)**:
   - Instructions, as well as word size, memory address modes, processor registers, and data type, etc,
2. **Microarchitecture**:
   - It addresses the implementation of ISA.
3. **Multicore design issues**:
   - Cache coherency.
   - Internal buses interfacing with the memory and the peripherals.



- Block diagram of a multi-core processor (**SoC**) with internal multi-level cache memories.
- The cores are connected to the main memory through internal buses.

SoC: System on a Chip

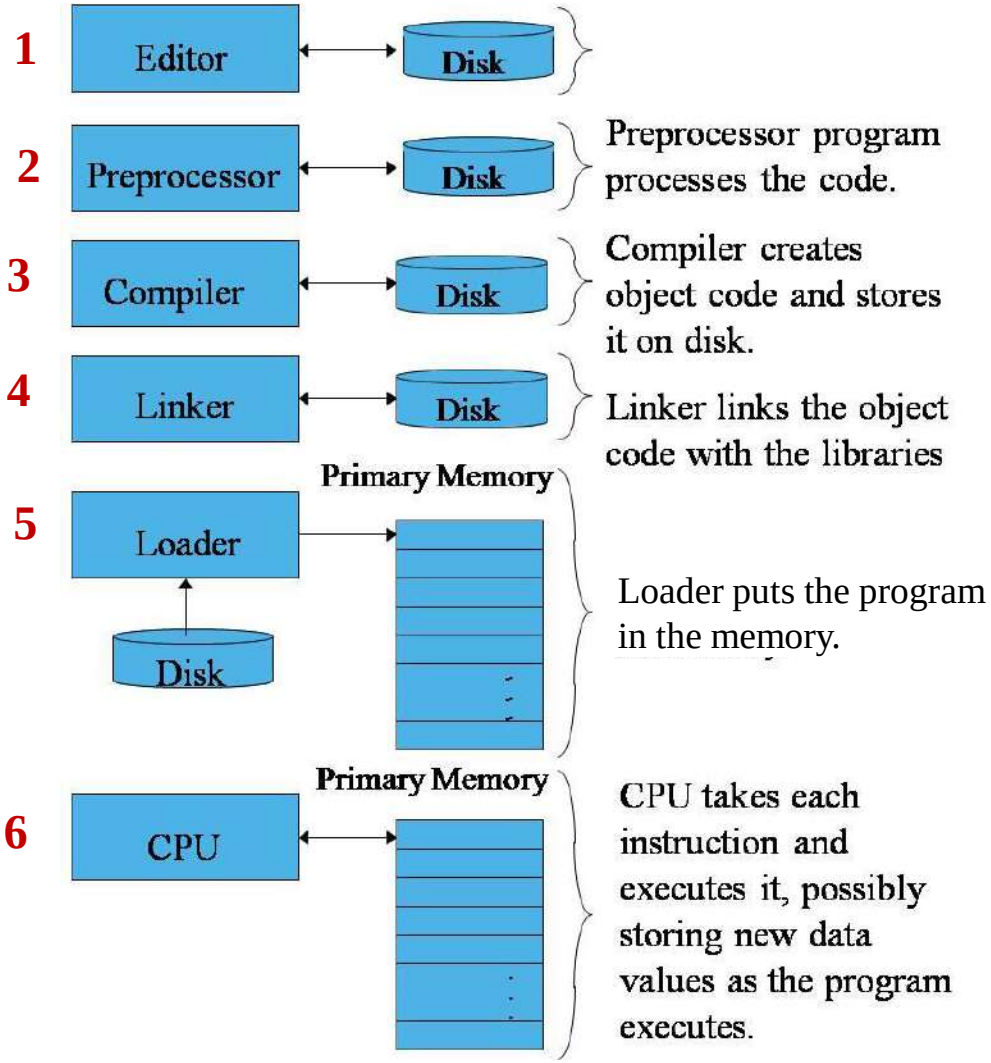
Note: Computer architecture also includes system level components, such as memory, peripherals etc.

SW Development Tools

Phases of SW Program Development

1. **Edit** – Source files (**.c and .h**)
2. **Preprocess** – Part of compilation (include files)
3. **Compile** – Get the **.obj** file of each source file
4. **Link** – Link libraries and your own object files
5. **Load** – Load the executable (along with static data)
6. **Execute** – Run the program on the CPU

SW Program Development Environment



1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute

Program to an Executable

From a C program to an Executable

```
int  count;

count++;
```

File: test.c

Snippet of a C program test.c

test.c is compiled using a C ***compiler***

```
ADR   R1, count   ; load the address of count in R1
LDR   R0, [R1]    ; copy the content of count into R0
ADD   R0, R0, #1  ; increment R0 by 1
STR   R0, [R1]    ; copy the incremented value to count
```

File: test.asm (a text file with assembly instructions)

This is a readable format of .o file

```
1010011010
```

File: test.o (a binary file)

Output from the ***compiler***

test.o is linked with lib files using a ***linker***

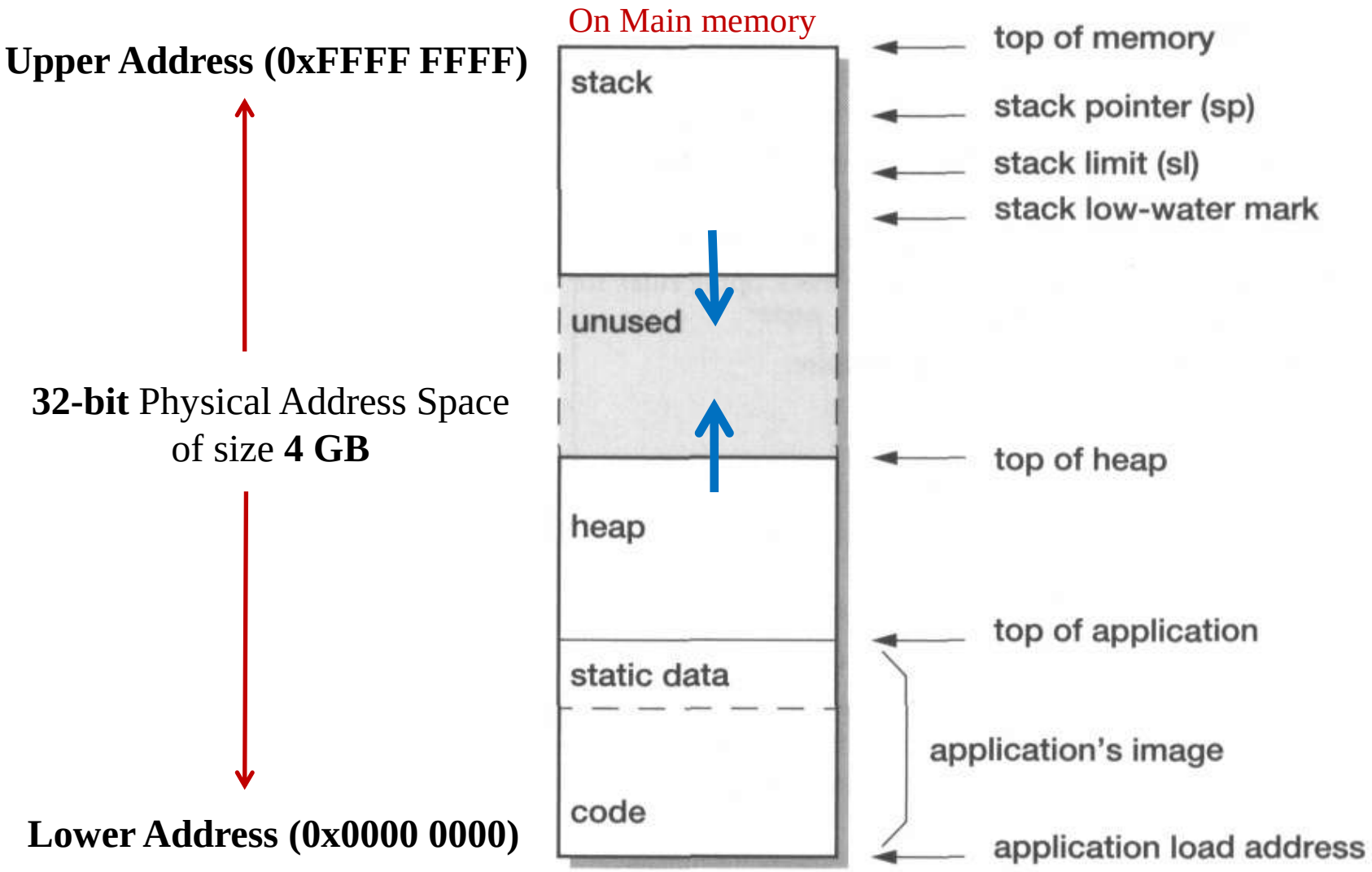
```
10111000011010
```

File: test.exe

Output from the ***linker***

Load the executable into memory using a ***loader*** before the execution.

Standard 32 bit Processor Address Space Model: An Example

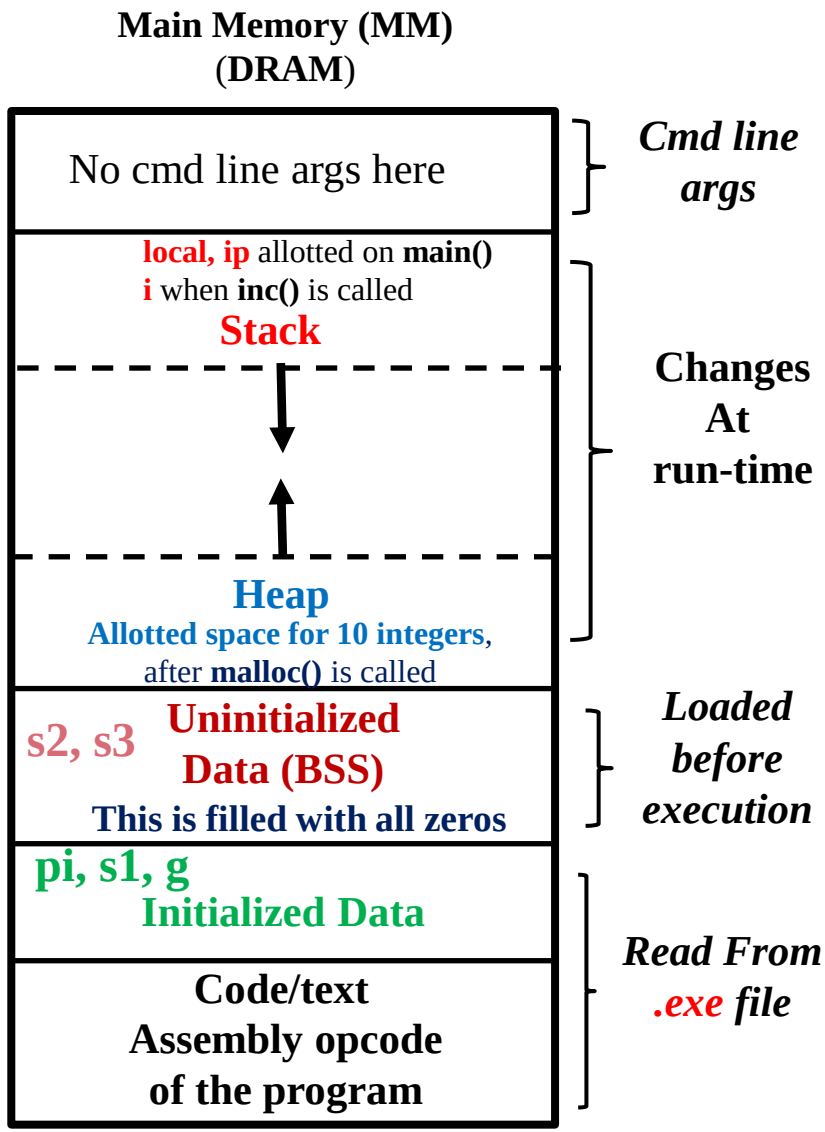


Let us now view the memory layout of a simple C program next …

Typical Memory Layout of C Program (Data)

```
#include <stdio.h>
#include <stdlib.h>
/* These global variables are not used here */
const float pi = 3.14;
static int s1 = 10;
static int s2;
int g = 2;

int inc(int i) {
    return i++;
}

main(void) {
    int local = 10; int *ip = NULL;
    static int s3;
    ip = (int *) malloc(sizeof(int) * 10);

    local = inc(local);

    free(ip); /* ip not used by the program */
    return 0;
} /* end of main() */
```

Note: OS prepares this program layout on MM by reading the executable from the hard disk, before giving the control to the program.



BSS: Block Started by Symbol

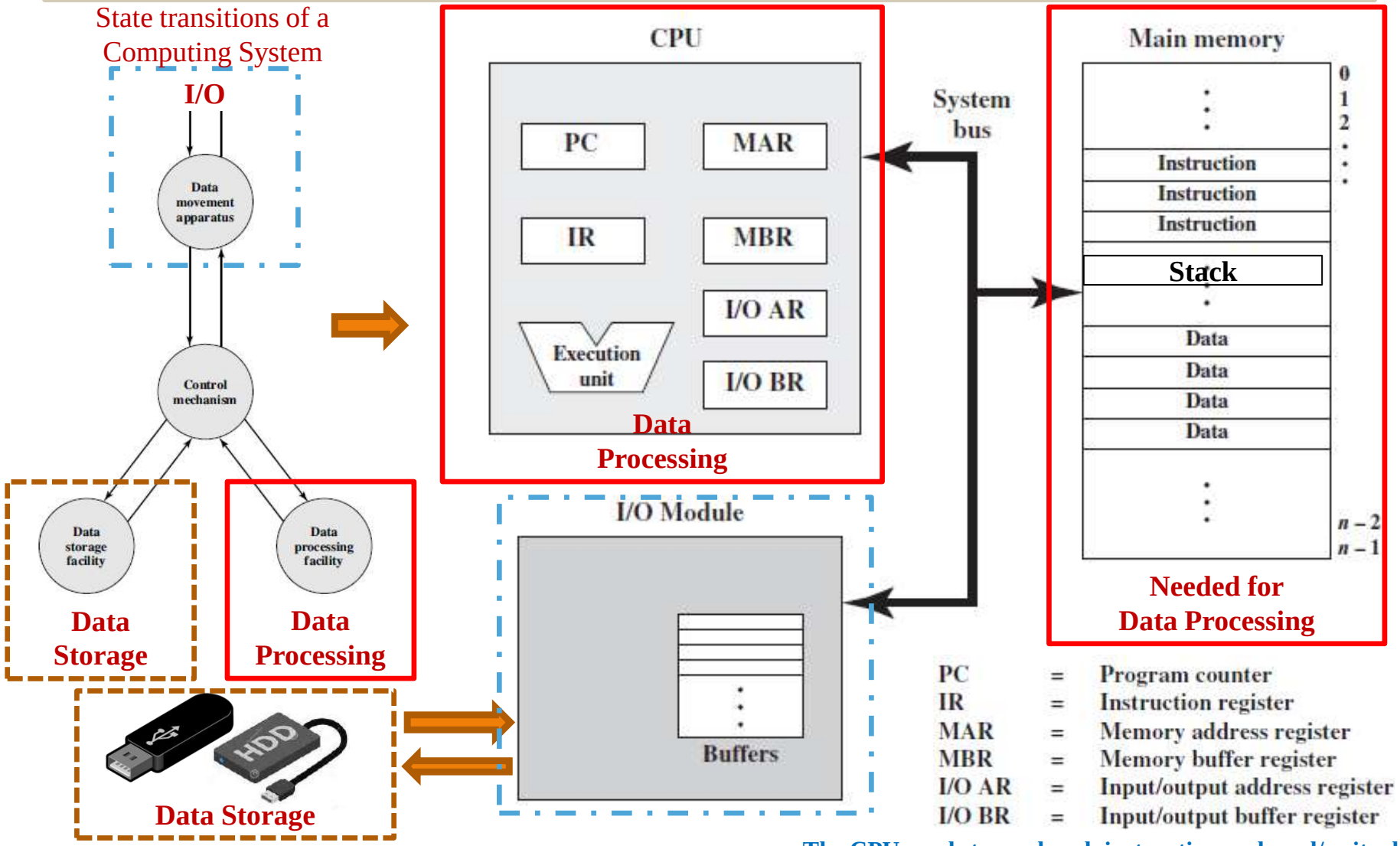
Architectural View of Computing System

Architectural View of a Computer System

(from the architecture point of view)

DRAM

State transitions of a Computing System



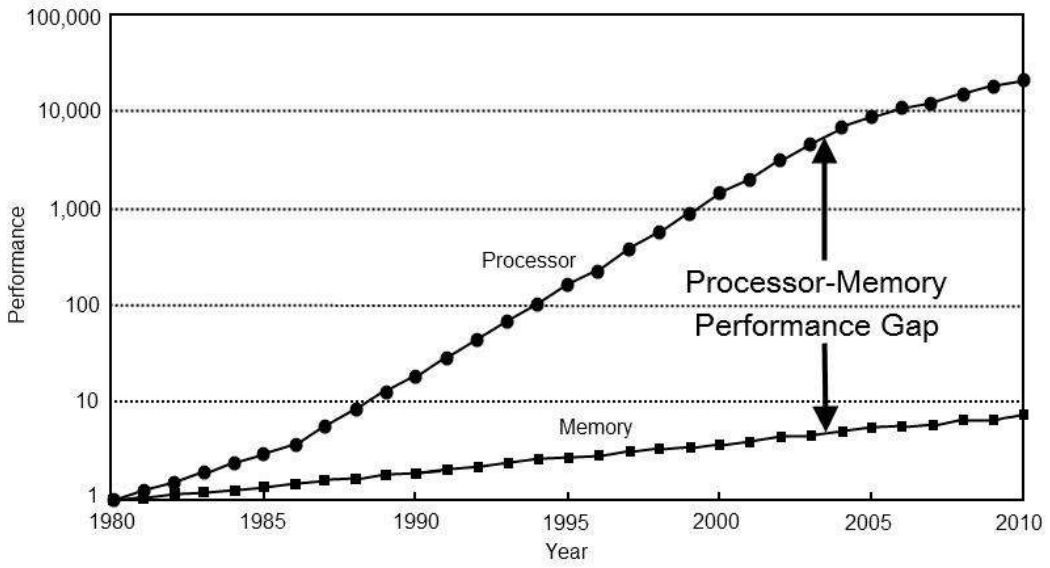
Control module coordinates the interactions between all the modules.

The CPU needs to read each instruction and read/write data from/to the memory, is there an issue with it?

CPU-DRAM Performance Gap

CPU-DRAM Performance Gap

- The **performance** gap between **CPU** and **DRAM** is shown below:
  - **CPU** performance **increases** by **60% every year**
  - **DRAM** performance **increases** by less than **10% every year**



Next, let us see one important property of software that helps in reducing the impact on software efficiency due to this performance gap between CPU and memory …

Principle of Locality

Principle of Locality

- Programs tend to reuse **data** and **instructions** that are **closer** to **those** that have been **used recently**, or that were **recently referenced** themselves.
- **Temporal locality**: **Recently referenced** items are **likely** to be **referenced** in the **near future.**
- **Spatial locality**: **Items** with **nearby addresses** tend to be **referenced** more **often** with **higher probability.**

Sample code … ⇒ Identify: Temporal or Spatial locality?

```
int sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

- **For Data**
  - Access of **array** elements in succession — Spatial locality
  - Access of **sum** on each iteration — Temporal locality
- **For Instructions**
  - Access of instructions in a sequence — Spatial locality
  - Cycling through the loop repeatedly — Temporal locality

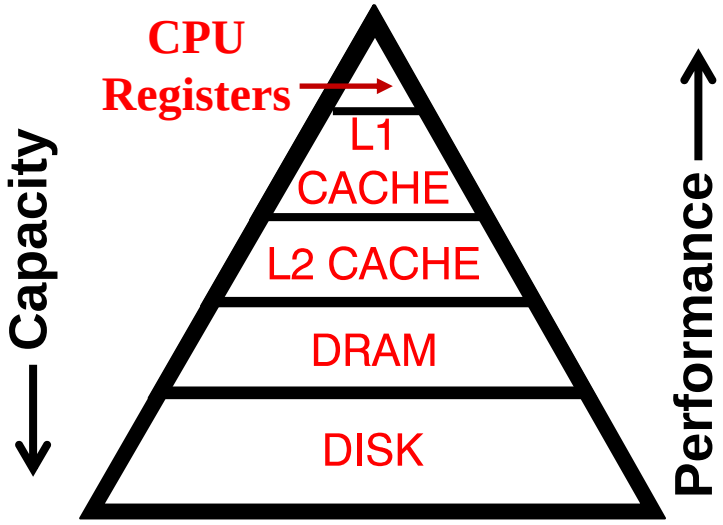
How can this property be leveraged to fill the gap between the speeds of CPU and memory? … let us see next …

Memory/Storage Hierarchies

Memory/Storage Hierarchies

- Balancing performance with cost
  - **Smaller** memories are faster but expensive.
  - **Larger** memories are slower but cheaper.
- Exploit locality to get the best of both worlds
  - Reuse of nearness of accesses.
  - Results in **most accesses** using the **smaller** and **faster** memories.
  - Along with **larger** memory available which costs **cheaper.**

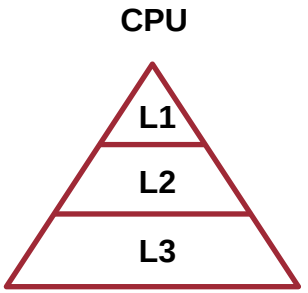
This enables sequential programs to run faster on a processor system …



There other means of improving the performance of program execution by parallel processing techniques using multi-core processors.
our **RP2040** is itself a dual core processor.

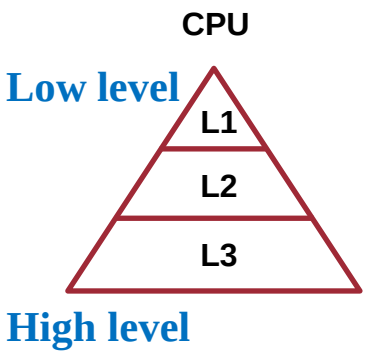
Three Properties of Memory Levels

- **Inclusion**: Any data that is part of a lower level (closer to the processor) needs to be present at the higher level
- **Coherence** (consistency): Multiple copies of the same data are available at each level (Registers, L1 cache, L2 cache, Main Memory, etc.)
  - All copies need to be identical or maintained consistent
- **Locality of Reference:**
  - **Temporal**: Will be used in the near future
  - **Spatial**: Adjacent data are likely to be used often
  - **Sequential**: Execution of instructions



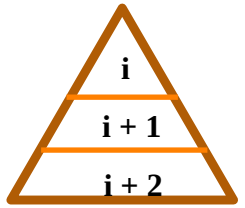
Properties and Issues with Memory Levels

- The caches which are farther to the CPU are of larger sizes than the ones that are closer to the CPU
  - Size (L3) > Size (L2) > Size (L1)
- The sizes of caches are always in multiples of powers of two
  - Sizes of higher-level caches would also be multiples of lower-level cache sizes
  - Example: Size (L1) = 32 KB, Size (L2) = 256 KB, Size (L3) = 2 MB.
- If some code or data has been brought into L1 cache, the same copy of code or data will also be present at all the higher-level caches
  - Having multiple copies of the same data has associated issues
  - Maintaining consistency across multiple copies of data during the execution of a program is a challenge
- This issue is called '**cache coherency**', which is to be addressed in multiprocessor systems



Quiz: Relationship between Memory Levels

- Access time (t_i) : $t_i < t_{i+1}$
- Cost per Byte (c_i) : $c_i > c_{i+1}$
- Memory size (s_i) : $s_i < s_{i+1}$
- Transfer Bandwidth (b_i) : $b_i > b_{i+1}$
- Unit of transfer (x_i) : $x_i < x_{i+1}$
- Frequency of access (f_i) : $f_i > f_{i+1}$



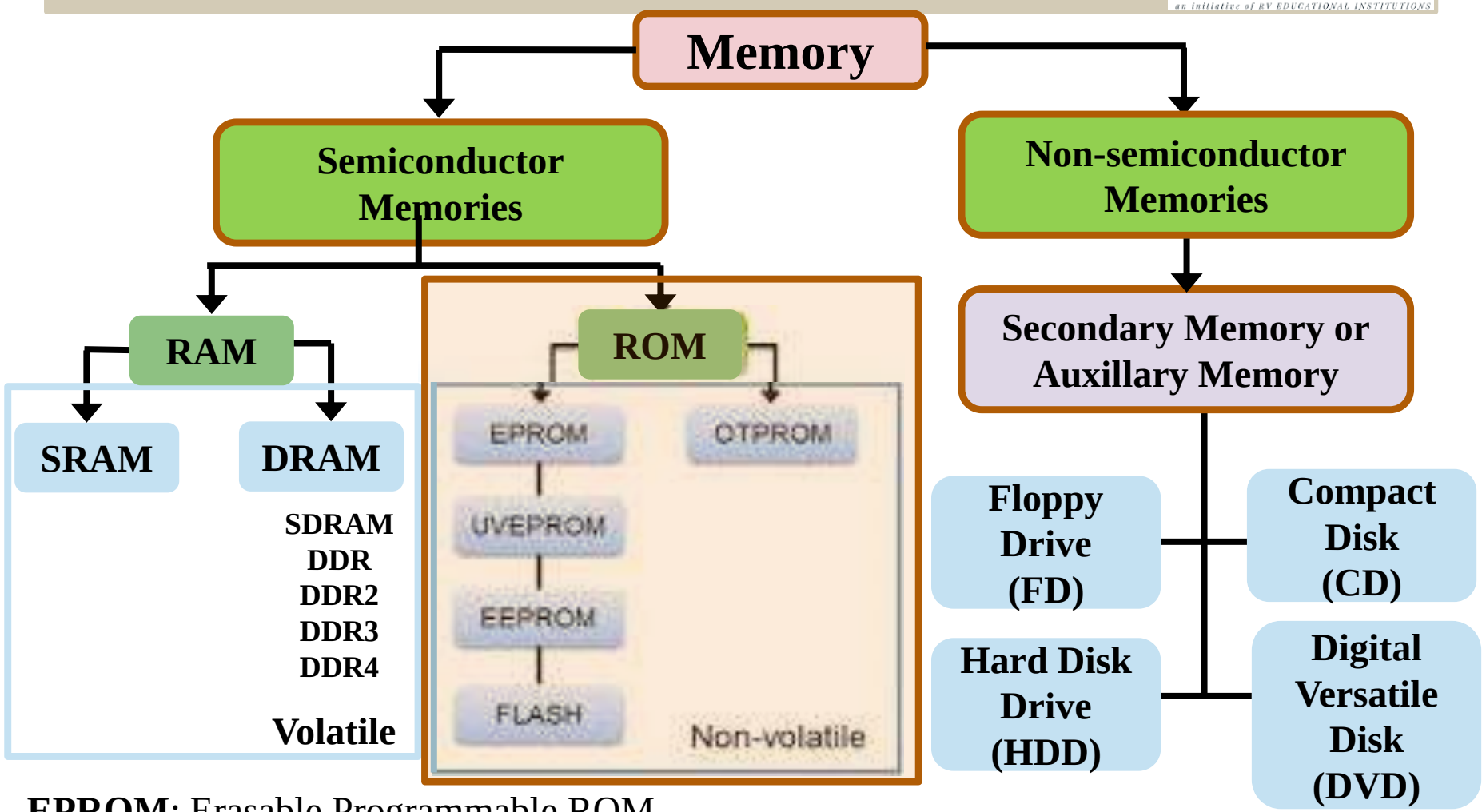
Notes: **a) i** is **closer** to the **processor** than **i+1**

b) Unit of **Bandwidth** is **Bytes/Second**

Access time: Time taken to access data from the memory

Different Memory Types

Memory Classification



EPROM: Erasable Programmable ROM

UVEPROM: Ultra-violet Erasable PROM

EEPROM: Electrically Erasable PROM

FLASH: Flash Memories

OTPROM: One Time PROM

Volatile and Non-volatile Memory

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Volatile:

- **Contents** stored in the **memory** are **lost** when the **power** is **withdrawn**
- The data in it will be cleared as soon as the laptop is shutdown
- The advantage of RAM is that, it can be directly accessed by CPU, much faster than non-volatile memories

Non-volatile:

- **Retains** the **contents stored** even if **power** is **switched off**
- **Non-volatile memory** is typically used for the task of **secondary storage**, or **long-term** persistent **storage**.

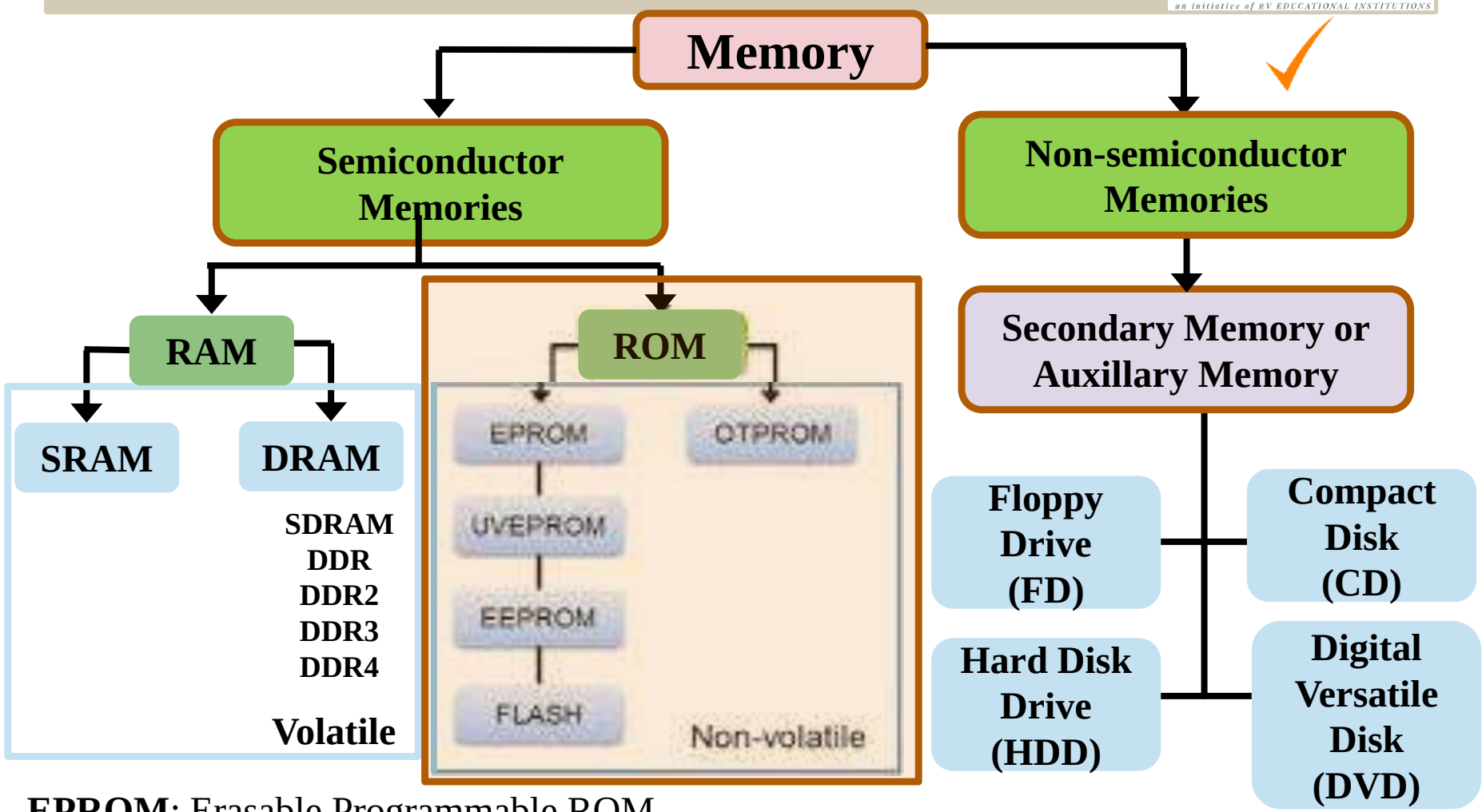
Semiconductor Vs Non-semiconductor Memories

Semiconductor Memories	Non-semiconductor Memories

• **Semiconductor technologies** are **used** for building it
• **Random access** capability
• **Mostly Volatile** in nature
  ◦ **Flash drives** are an exception, which are **non-volatile** and **erasable**
• **Access time** is much **lower**
  ◦ It take less time to read from it
• **Closer to CPU**, whereas **SRAM** is built within the CPU chip itself
• The size is in terms of **G Bytes** (**10^9**)
	• **Magnetic** and **optical** technologies used
	• Normally **sequential access**
	• **Non-volatile**
	• Has **higher access time**
	  ◦ It takes more time to read from it
	• **Farther** from **CPU** and built as a separate unit
	• Can be of much larger sizes, in terms of **Tera Bytes** (**10^{12}**) and more

Read-Only Memories (ROM)

Memory Classification



EPROM: Erasable Programmable ROM

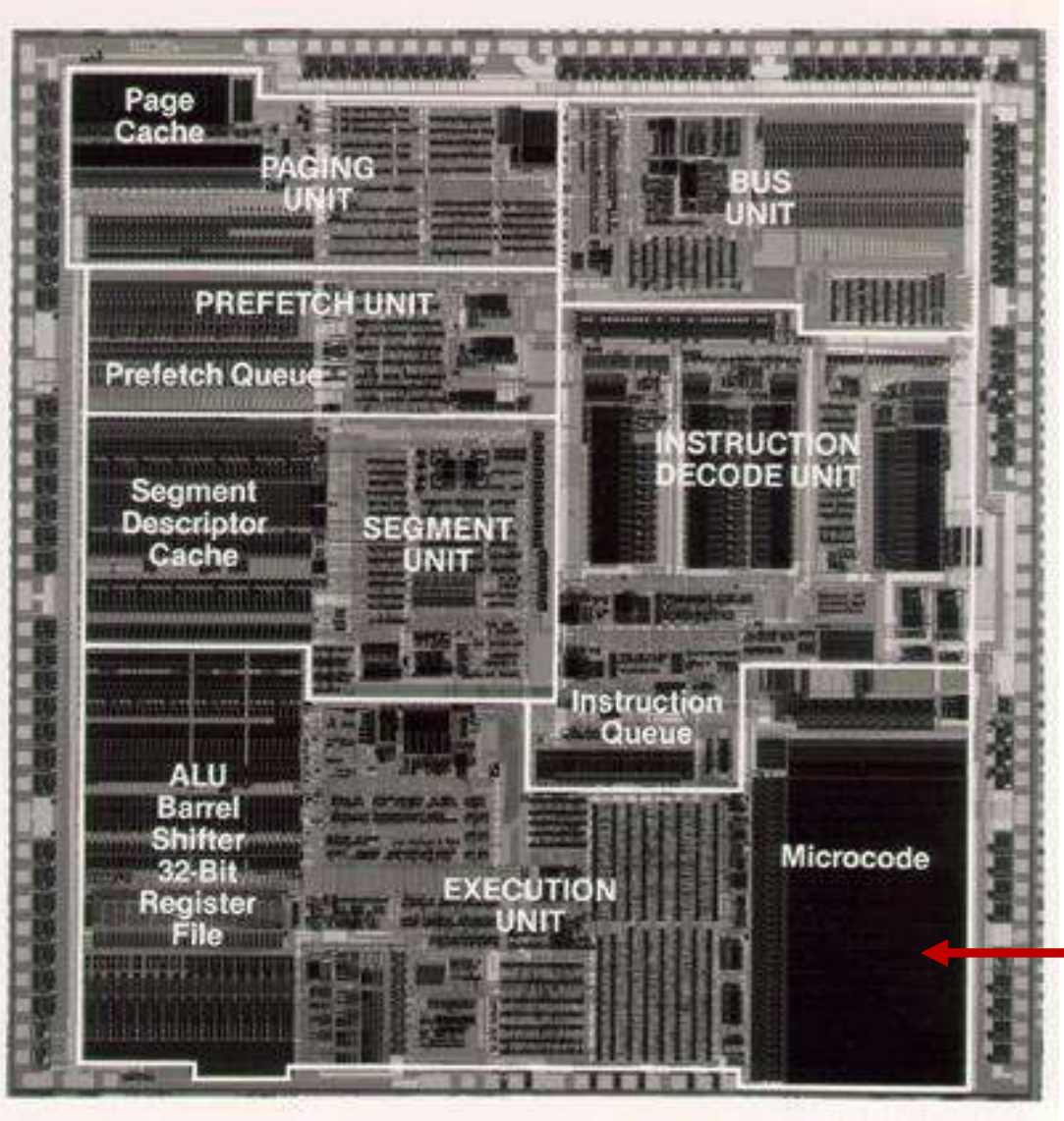
UVEPROM: Ultra-violet Erasable PROM

EEPROM: Electrically Erasable PROM

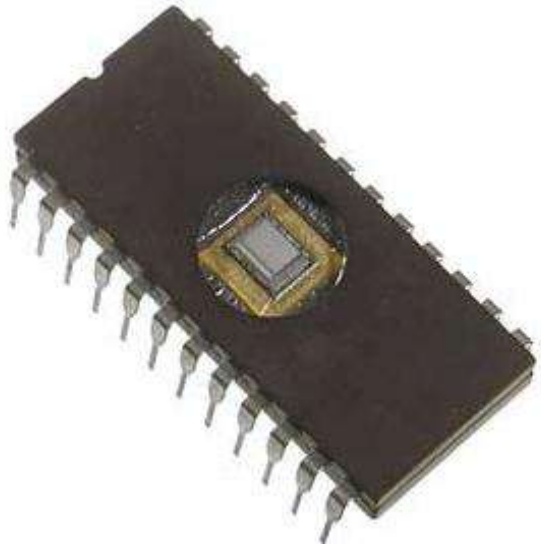
FLASH: Flash Memories

OTPROM: One Time PROM

Inside a Microprocessor: Sample



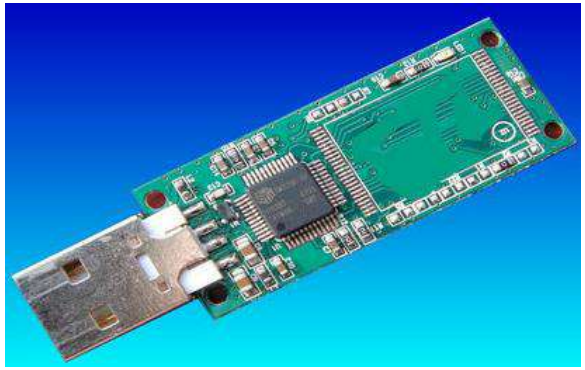
Types of ROM Memories

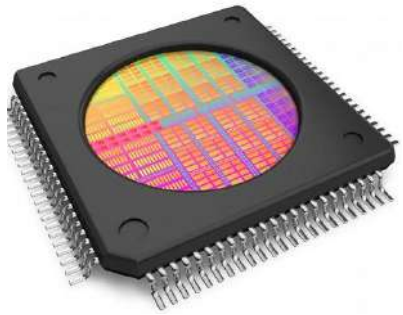


UV Erasable PROM

8 pin,
2-wire
Serial
Interface
64kx8 bits
size

Electrically Erasable PROM (EEPROM)





Flash memory: USB Memory Stick

ROM and PROM

What is a ROM?

- **ROM** is a Programmable Logic Device (**PLD**)
- **Binary** information is **hard-wired** inside a **ROM**, during the manufacturing of the device
  - This process is referred to as **programming** the **device**
  - Process is a set of **hardware connections** to store binary values
  - Once it is done it cannot be modified

Types of ROM: 1. ROM

- Here, the data is **wired** into the chip during the **HW fabrication process**
- As the name suggests the **first versions** of **ROM** were **read-only**
  - While it was **possible to read** a **ROM**, it was **not possible** to **update** them or **write** any **new data** into them
- An important **application** of **ROMs** is to **store microprograms** or **microcode** inside **microprocessors**
  - Which we studied in the **CPU design**, to **store** a sequence of **µ-ops** to **execute** various **assembly instructions** in a CPU

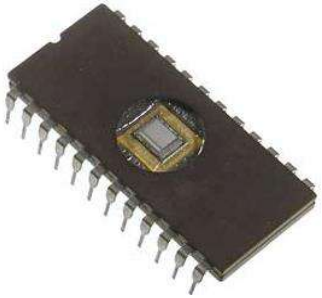
Types of ROM: 2. PROM

- Like the ROM, the **PROM** is **non-volatile** and may be **written** into **only once**
- For the **PROM** the **writing process** is done **electrically**
  - Not wired into the chip during manufacture, like in ROM
- Writing (once) may be performed by the **supplier** or the **user** at a time later than the original chip fabrication
  - The contents of PROM cannot be written more than once
- Special equipment is required for the writing or "programming" process
- They are also called **One Time Programmable**
  - **OTP NVM** (non-volatile memories)
- PROMs provide flexibility and convenience and expensive than ROM
- ROM remains attractive for high volume production runs

Quiz 1: ROM Vs PROM – Yes or No

- **ROM**: Data is wired into it during manufacture: **YES**
- **ROM**: Writing into this is possible by the user: **NO**
- **PROM**: Writing is same as programming the chip: **YES**
- **PROM-OTP**: Cannot be written more than once: **YES**
- **PROM**: Programming is done during manufacture: **NO**
- **PROM**: It is more expensive than ROM: **YES**
- **ROM**: It can be used for storing µ-ops in CPUs: **YES**

UV Erasable PROM



160D0WQ
EEPROM Chip

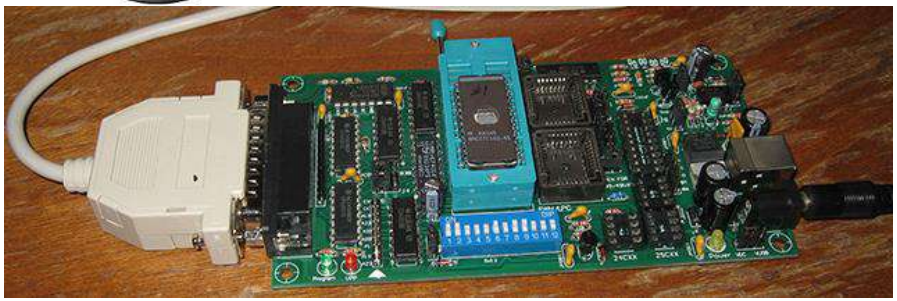
EPROM and EEPROM

EPROM: Programming Sequence

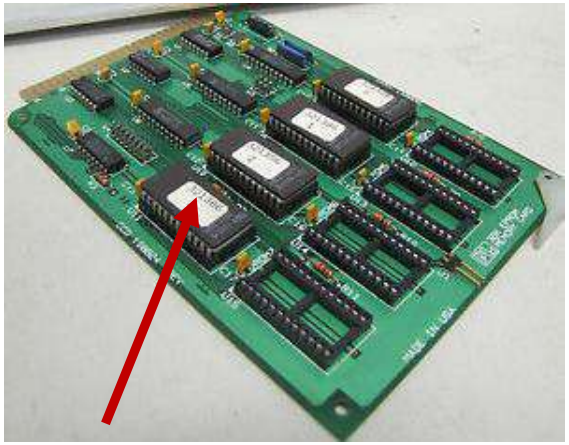
1. ERASE the entire chip

EPROM programmer Connected to a **PC**

2. PROGRAM it



3. USE it

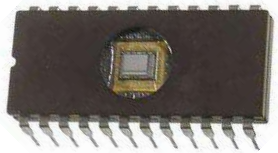


- Keep the **EPROM** in an **UV eraser** for **20 minutes** to **erase all** the contents of it. After erasing, the whole memory will be **filled** with **1s**
- Then, using a **PROM programmer, write specific contents** into **PROM**
- **Put** it back into the **socket** on the **board** to **use** it as **normal memory chip** to read the program and /or data from it

PROMs are **covered** with **stickers** to **avoid accidental erasing** due to **exposure to light**

Types of ROM: 3. EPROM

- **Erasable programmable read-only memory** is **optically (UV) erasable**
- EPROMs are **read** and **written electrically**, similar to PROM (used as normal memories on the circuit board)



- However, before a write operation, all the storage cells must be erased to the same initial state (**all 1s**) by **exposing** the packaged **chip** to **ultraviolet radiation**
- **Erasure** is performed by **shining UV** through the window on the chip for a period of around **20 minutes**
- Thus, **EPROM** can be **erased multiple times**
- **EPROM** is **more expensive** than **PROM** and has **multiple update** capability

Types of ROM: 4. EEPROM

- A more attractive form of ROM is **electrically erasable programmable read-only memory** (**EEPROM**)
- This is a read-mostly memory that can be written into at any time, without erasing the entire prior contents
  - Updating only a byte or a set of bytes is possible
- The **write operation** takes considerably **longer time** than the read operations, in the order of **several 100 µsecs**
- EEPROM combines the advantage of non-volatility with the flexibility of being updatable in place (in-circuit)
  - Using ordinary bus control, address and data lines
- EEPROM is more expensive than EPROM and less dense
  - i.e., EEPROM supports fewer bits per chip than EPROM

Quiz 2a: EPROM Vs EEPROM – Yes or No

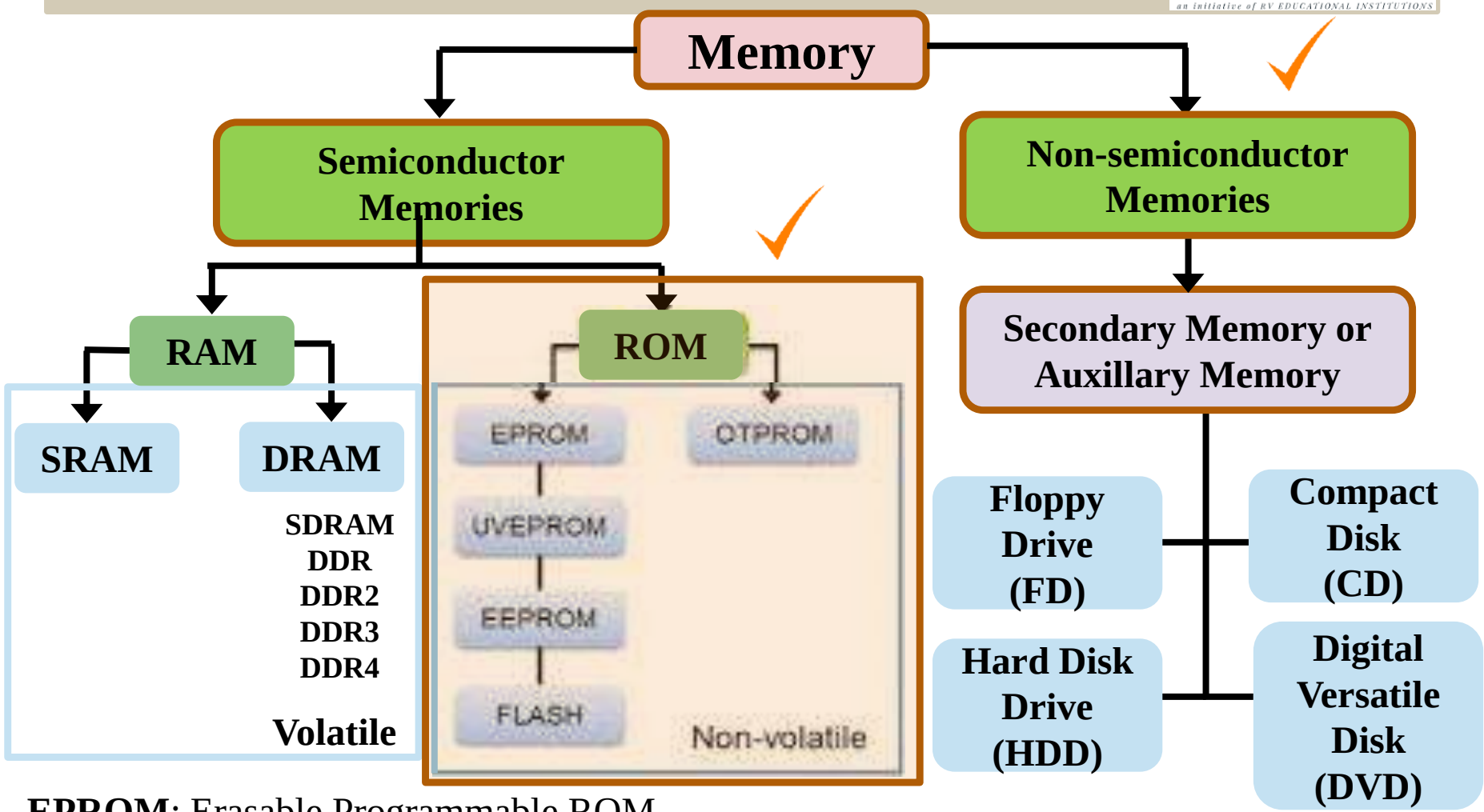
- **EPROM**: UV is used to erase the contents: **YES**
- **EPROM**: It is possible to erase part of the chip: **NO**
- **EEPROM**: Erasing is done optically: **NO**
- **EPROM**: Reading and writing are done electrically: **NO**
- **EPROM**: Can be written only after erasing all data: **YES**
- **EEPROM**: Erasing is easier than EPROM: **YES**
- **EEPROM**: Writing is faster than reading from it: **NO**
- **EEPROM**: It is more expensive than EPROM: **YES**
- In general **writing** is **slower than reading** from **them** on **all types of ROMs**: **YES**

Quiz 2b: EPROM Vs EEPROM – Yes or No

- **EPROM**: Can be erased without removing it from the circuit where it is used as memory: **NO**
- **EPROM**: Programming them actually involves only writing zeros into specific locations: **YES**

Random Access Memory (RAM)

Memory Classification



EPROM: Erasable Programmable ROM

UVEPROM: Ultra-violet Erasable PROM

EEPROM: Electrically Erasable PROM

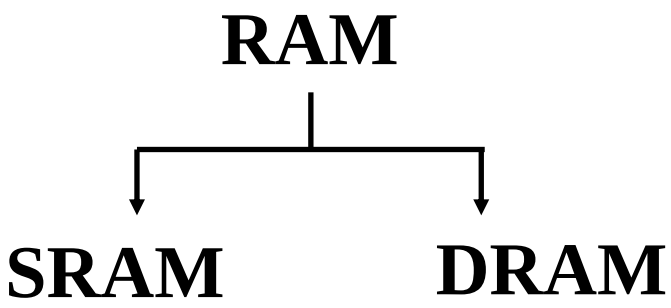
FLASH: Flash Memories

OTPROM: One Time PROM

What is Random Access Memory (RAM)?

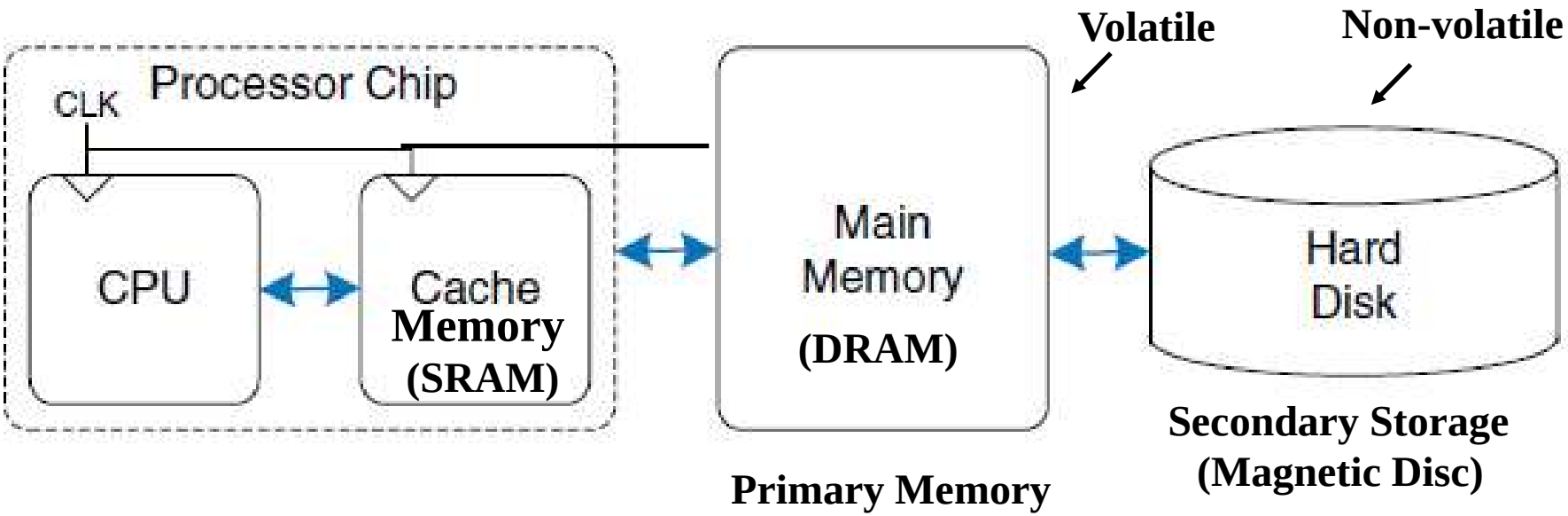
- The time taken to **read** or **write** a data into **any location** of the **RAM** takes the **same amount of time**
  - It **does not vary** with respect to the **location of the data**
- All locations of a RAM memory takes the same amount of time for read/write
- In contrast, with other direct-access data storage media such as hard disks, CD, DVD and the older magnetic tapes and drum memory,
  - The time required to read and write data items varies significantly depending on their physical locations on the recording medium

Major Types of RAM



- Computer memories are primarily built from D**ynamic RAM** (**DRAM**) and **Static RAM** (**SRAM**)
- **They** have **different construction** and **properties** but **both** types are **volatile**
  - Which means that they **lose** the **stored information** after the **power** is **shut off**

Use of SRAM and DRAM



- The processor first seeks data in a small but fast cache that is usually located on the same chip.
- If the data is not available in the cache, the processor then looks in main memory.
- If the data is not there either, the processor fetches the data from hard disk

Quiz 1: Hibernate or Sleep modes in Laptop

- What **happens** when the **laptop** is on **Hibernate** or **Sleep mode?**
- **Both** are **power-saving states**
- **Which** takes **more time** to **power ON?** **Hibernate**
- **What happens** when you put **laptop** to **sleep?**
- **Sleep: Power** to **display** and other **peripherals** is **switched off,** but the **main memory** is **powered.** It **retains** the **contents** of the **currently running programs,** so it can **restart quickly** from the **sleep** mode**.**
- **Hibernate:** The **contents** of the **main memory (RAM)** is **saved into** the **hard disk** and **RAM** is also **powered off.** It **takes more time** because **OS** has to **copy back** the **program contents** from the **hard disk back to RAM** before **start running** them**.**